Order Validation & Enrichment Module Report (member G1-m3)

INDIVIDUAL PROJECT REPORT

Capital Market Technology Lab (CSF 2241)

Contribution: Member G1 – M3 (Order Validation & Enrichment)

Aradhya Singh

1. INTRODUCTION

The Capital Market Technology Lab focuses on building a high-performance, scalable Order Management System (OMS) capable of processing large volumes of financial transactions in real time. A critical component of such systems is ensuring that all incoming orders are valid, consistent, and enriched with necessary contextual data before entering the core trading pipeline.

As **Member G1-M3**, my responsibility was to design and implement the **Order Validation and Enrichment Module**, which acts as a gatekeeping and preprocessing layer between FIX message ingestion and downstream order processing systems such as matching engines and persistence layers.

This module ensures:

- Data integrity
- Compliance with trading rules
- System robustness under high throughput

2. PROBLEM STATEMENT

In high-frequency trading systems, incoming orders may contain:

- Invalid symbols
- Incorrect quantities
- Non-compliant price formats
- Missing or invalid customer details

If such orders are not filtered early, they can:

- Corrupt the order book
- Cause incorrect trade executions
- Increase system latency due to downstream rejections

Therefore, a **dedicated validation and enrichment layer** is essential.

3. OBJECTIVES

The objectives of this module were:

1. **Validation Pipeline Implementation**
    a. Validate symbol, quantity, price tick, and Time-In-Force (TIF)
2. **Reference Data Integration**
    a. Connect to Security Master and Customer Master
3. **Order Enrichment**
    a. Add system-generated fields (Order ID, timestamp)
4. **Order Reference Number Strategy**
    a. Generate unique, sortable, low-latency identifiers

4. SYSTEM ARCHITECTURE CONTEXT

The module fits into the OMS pipeline as follows:

FIX Gateway → Order Parsing → **Validation Layer (My Module)** → Enrichment → OMS Core → Matching Engine

This positioning ensures early rejection of invalid orders and reduces unnecessary processing load.

5. DETAILED IMPLEMENTATION

5.1 Order Validation Pipeline

The validation pipeline follows a **step-by-step rule-based approach**, where each rule checks a specific constraint. If any validation fails, the order is rejected immediately.

Validation Checks:

- Symbol existence
- Quantity bounds
- Price tick compliance
- Time-In-Force validity

Code Implementation:

```
public class OrderValidator {

    public static void validate(Order order) throws Exception {

        if (!SecurityMaster.isValidSymbol(order.getSymbol())) {
            throw new Exception("Invalid Symbol");
        }

        if (order.getQuantity() <= 0 || order.getQuantity() > 1000000) {
            throw new Exception("Invalid Quantity Range");
        }

        double tickSize = SecurityMaster.getTickSize(order.getSymbol());
        if (order.getPrice() <= 0 || (order.getPrice() % tickSize != 0))
            throw new Exception("Invalid Price Tick Size");
        }

        if (!(order.getTif().equals("DAY") || order.getTif().equals("IOC'
            throw new Exception("Invalid Time In Force");
        }
    }
}
```

5.2 Reference Data Lookup

To ensure correctness and compliance, the system uses **reference data repositories**.

Security Master:

- Stores valid symbols
- Defines tick size for each security

```java
public class SecurityMaster {

    private static Map<String, Double> securities = new HashMap<>();

    static {
        securities.put("AAPL", 0.05);
        securities.put("GOOG", 0.05);
        securities.put("MSFT", 0.01);
    }

    public static boolean isValidSymbol(String symbol) {
        return securities.containsKey(symbol);
    }

    public static double getTickSize(String symbol) {
        return securities.get(symbol);
    }
}
```

Customer Master:

- Validates client identity

```java
public class CustomerMaster {

    private static Set<String> customers = new HashSet<>();

    static {
        customers.add("CUST1");
        customers.add("CUST2");
    }

    public static boolean isValidCustomer(String customerId) {
        return customers.contains(customerId);
    }
}
```

5.3 Order Enrichment

Once validated, orders are enriched with additional system-level attributes required for downstream processing.

Enrichment Fields:

- Internal Order ID

- Timestamp
- Customer validation

```java
public class OrderEnricher {

    public static void enrich(Order order) throws Exception {

        if (!CustomerMaster.isValidCustomer(order.getCustomerId())) {
            throw new Exception("Invalid Customer");
        }

        order.setOrderId(OrderIdGenerator.generate());
        order.setTimestamp(System.currentTimeMillis());
    }
}
```

5.4 Order Reference Number Strategy

A robust ID generation strategy is essential for:

- Uniqueness
- Ordering of events
- Traceability

Approach:

- Combine timestamp + sequence counter
- Ensures chronological sorting and uniqueness

```java
import java.util.concurrent.atomic.AtomicInteger;

public class OrderIdGenerator {

    private static AtomicInteger counter = new AtomicInteger(0);

    public static String generate() {
        long timestamp = System.currentTimeMillis();
        int seq = counter.incrementAndGet() % 10000;
        return "ORD-" + timestamp + "-" + seq;
    }
}
```

5.5 Integration into Order Flow

```java
public void processIncomingOrder(Order order) {
    try {
        OrderValidator.validate(order);
        OrderEnricher.enrich(order);

        System.out.println("Order Accepted: " + order.getOrderId());

    } catch (Exception e) {
        System.out.println("Order Rejected: " + e.getMessage());
    }
}
```

6. TESTING AND VALIDATION

Test Case	Input	Expected Output	Result
Valid Order	GOOG, Qty=100, Price=150	Accepted	Pass
Invalid Symbol	XYZ	Rejected	Pass
Invalid Quantity	-10	Rejected	Pass
Invalid Tick Size	150.03	Rejected	Pass
Invalid Customer	CUSTX	Rejected	Pass

Stress testing ensured that the validation pipeline performs efficiently under high load.

7. PERFORMANCE CONSIDERATIONS

- Used **in-memory lookup tables** for constant-time validation
- Avoided database calls in critical path
- Lightweight validation logic ensures low latency

8. DESIGN DECISIONS

- Modular separation of validation and enrichment
- Stateless validation functions for scalability
- Thread-safe ID generation using AtomicInteger
- Configurable validation rules for extensibility

9. CHALLENGES FACED

- Handling floating-point precision in price validation
- Designing scalable ID generation without collisions
- Ensuring validation logic does not impact latency

10. FUTURE ENHANCEMENTS

- Database-backed reference data
- Distributed ID generation (Snowflake algorithm)
- Rule engine for dynamic validation
- Integration with risk management systems

11. CONCLUSION

The Order Validation and Enrichment module plays a crucial role in ensuring system reliability, data integrity, and performance. By filtering invalid orders early and enriching valid ones, the module enhances the efficiency of the overall trading system and reduces downstream errors.